

## LABORATORIO N° 05

### Desarrollo de Software



DOCENTE:  
Bestard Aroche, Yunion

CURSO:

Desarrollo de Aplicaciones  
Empresariales  
4 - C24 - Sección CD

## DESARROLLO DE APLICACIONES EMPRESARIALES:

### Capacidades

1. **Identificar la funcionalidad del administrador de Django y su papel dentro de una aplicación empresarial.**
2. **Registrar en el Django Admin modelos relacionados (1:1, 1:N y N:M) mediante ModelAdmin, personalizando list\_display, search\_fields y list\_filter.**
3. **Implementar el administrador y personalizar su plantilla en una aplicación Web en Django mediante Inlines (TabularInline y StackedInline) para exponer relaciones desde una sola pantalla.**

#### I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

#### II. Fundamento teórico

- Revise el material de la Semana 5 (Administrador de Django: registro de modelos, ModelAdmin, personalización y relaciones mediante Inlines) antes del desarrollo del laboratorio.

#### III. Normas empleadas

- No aplica

#### IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada en el laboratorio de la Semana 4 (Models con relaciones OneToOneField, ForeignKey y ManyToManyField con through) y, cuando corresponda, la investigación propia ampliada en la Parte 2 de dicho laboratorio.

#### V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

#### VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código en admin.py y del panel del Django Admin en el navegador, mostrando explícitamente el registro de modelos, la personalización (list\_display, search\_fields, list\_filter) y los Inlines implementados.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

## VII. Procedimiento

En la Semana 4 se incorporaron relaciones entre Models (OneToOneField, ForeignKey y ManyToManyField con through) sobre la aplicación ya persistente desde la Semana 3. En esta sesión no se agregan Models nuevos ni relaciones nuevas: se incorpora el Django Admin, la interfaz de administración que Django genera automáticamente a partir de los Models ya registrados, permitiendo gestionar esos datos relacionados sin escribir una View propia.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de la Semana 4 y registra sus modelos (incluidas las tres relaciones) en el Django Admin, personalizándolo con ModelAdmin e Inlines. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades ampliadas en la Parte 2 del laboratorio de la Semana 4), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que el Django Admin no reemplaza al Model ni al ORM: los reutiliza. El mismo Model que ya definió y las mismas relaciones que ya implementó son las que el Admin expone automáticamente, y el estudiante decide cómo presentarlas (list\_display, search\_fields, list\_filter) y cómo editar relaciones desde una sola pantalla (Inlines).

### Staff autenticado

- URL /admin/...
- Django Admin autentica y enruta
- ModelAdmin
- Manager / QuerySet
- Django ORM
- SQLite
- Template (generado automáticamente por el Admin)
- Response

Al finalizar el laboratorio, el estudiante deberá ser capaz de registrar modelos relacionados en el Django Admin, personalizar ModelAdmin con list\_display, search\_fields y list\_filter, y exponer relaciones 1:1 y N:M mediante StackedInline y TabularInline, reconociendo además qué resuelve el Admin y qué seguirá requiriendo una View y un Template propios.

## Enunciados

### PARTE 1 — Administrar en el Django Admin la aplicación de la Semana 4

#### Ejercicio 1 — Recuperar la aplicación de la Semana 4

Abra la aplicación desarrollada en el laboratorio de la Semana 4 e identifique su entidad principal, las tres relaciones implementadas (OneToOneField, ForeignKey y ManyToManyField con through) y el estado actual de sus migraciones.

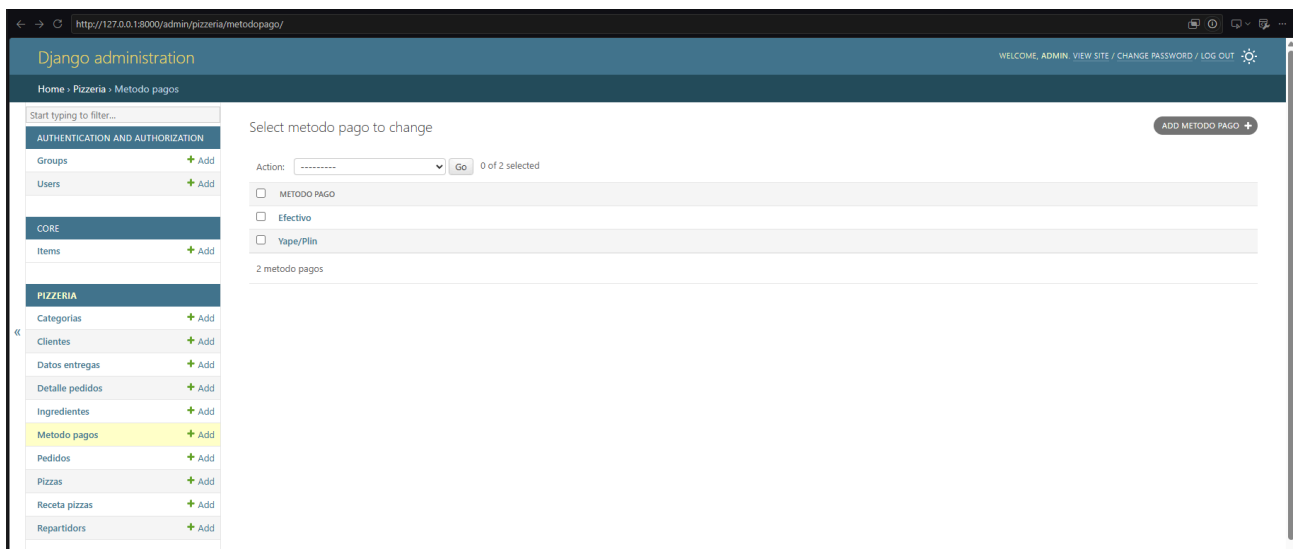
```
(.venv) PS C:\Users\jfacu\Desktop\DesarrolloAplicacionesEmpresariales\semana05\src> python manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
core
[X] 0001_initial
[X] 0002_alter_item_id
[X] 0003_alter_item_created_at_alter_item_description_and_more
pizzeria
[X] 0001_initial
[X] 0002_metodopago_descripcion_alter_cliente_nombre
[X] 0003_pedido_tipo_pedido_alter_pedido_repartidor
[X] 0004_alter_repartidor_options_repartidor_placa
[X] 0005_alter_ingrediente_unidad_medida
sessions
[X] 0001_initial
```

Consiste en abrir el proyecto desarrollado en el laboratorio anterior, identificando la entidad principal, las relaciones (OneToOneField, ForeignKey y ManyToManyField con through) y verificando mediante showmigrations que las migraciones estén aplicadas correctamente.

## Ejercicio 2 — Crear el superusuario

Ejecute `python manage.py createsuperuser` y cree una cuenta de administrador. Inicie sesión en `/admin/` y documente con una captura el panel inicial del Django Admin.

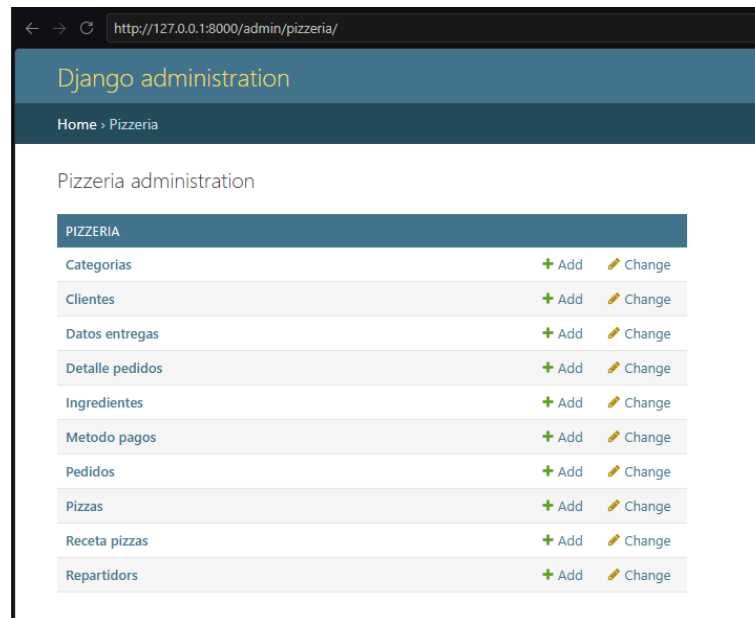
```
Username (leave blank to use 'jfacu'): jfacu
Email address:
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```



Trata sobre la ejecución del comando `python manage.py createsuperuser` en la terminal para crear la cuenta con privilegios administrativos que permitirá iniciar sesión en la ruta `/admin/` del sistema.

### Ejercicio 3 — Registrar los modelos en el Admin

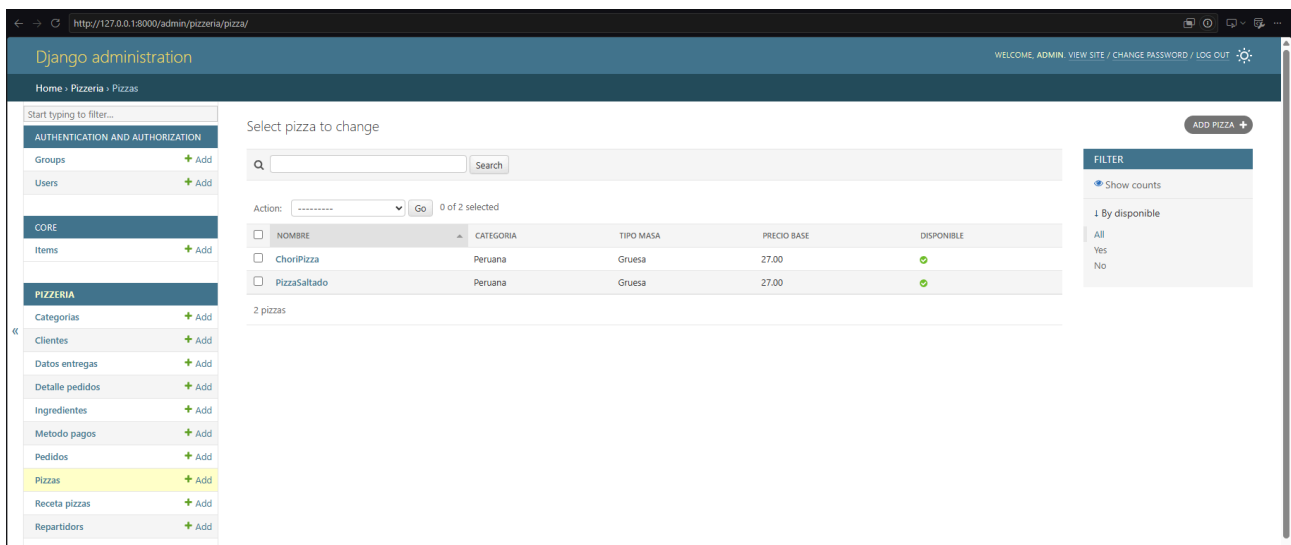
En `admin.py` de la app correspondiente, registre todos los modelos de la Semana 4 (la entidad principal y las entidades de las tres relaciones) mediante `admin.site.register()`. Verifique que todos aparecen en el panel.



Abarca el registro básico de todos los modelos de la aplicación en el archivo `admin.py` utilizando `admin.site.register()`, comprobando que aparezcan listados en el panel de administración.

### Ejercicio 4 — Personalizar con ModelAdmin

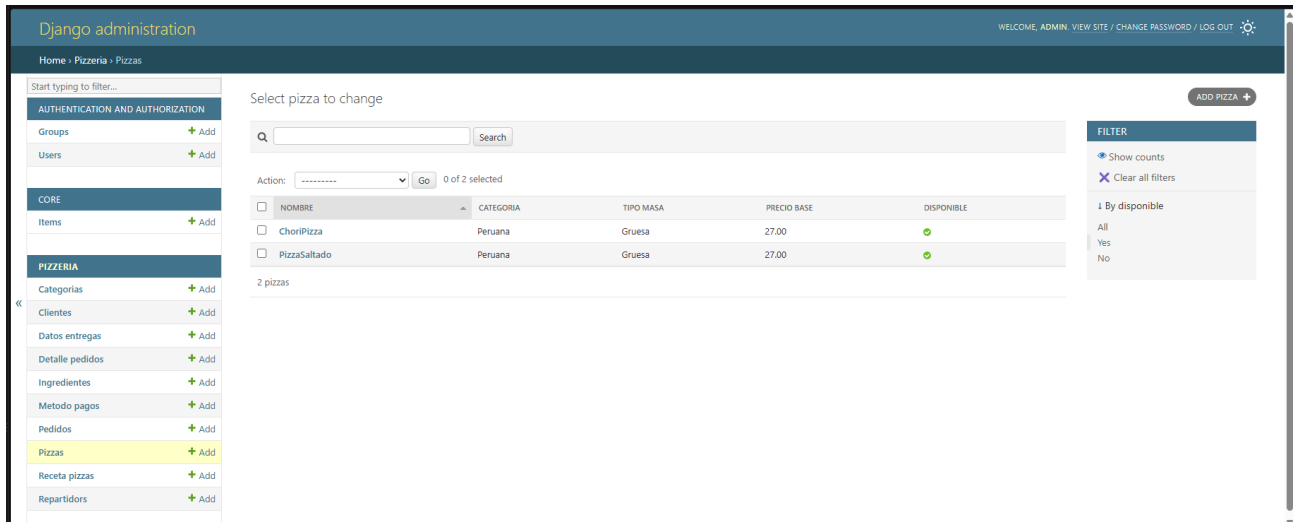
Reemplace al menos dos registros simples por una clase `ModelAdmin`, configurando `list_display` con los campos más relevantes de cada modelo y `search_fields` para al menos uno de ellos.



Consiste en reemplazar el registro simple de al menos dos modelos por clases personalizadas que hereden de `admin.ModelAdmin`, configurando `list_display` para mostrar columnas relevantes y `search_fields` para habilitar una barra de búsqueda.

## Ejercicio 5 — Agregar list\_filter

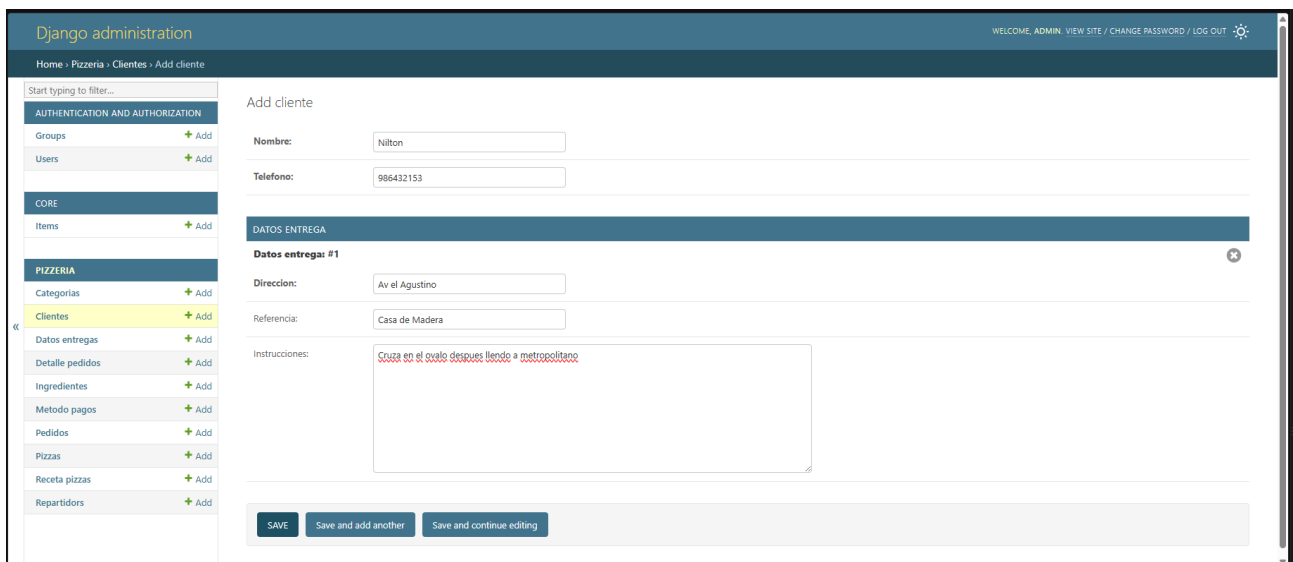
Configure list\_filter en al menos un modelo, usando un campo que tenga sentido filtrar (por ejemplo, una fecha, una categoría o un estado). Documente con captura el panel de filtros funcionando.



Implementa el atributo list\_filter dentro de una clase ModelAdmin usando campos como fechas, categorías o estados, lo que despliega un panel lateral de filtros en la interfaz del Administrador.

## Ejercicio 6 — Exponer la relación 1:1 con StackedInline

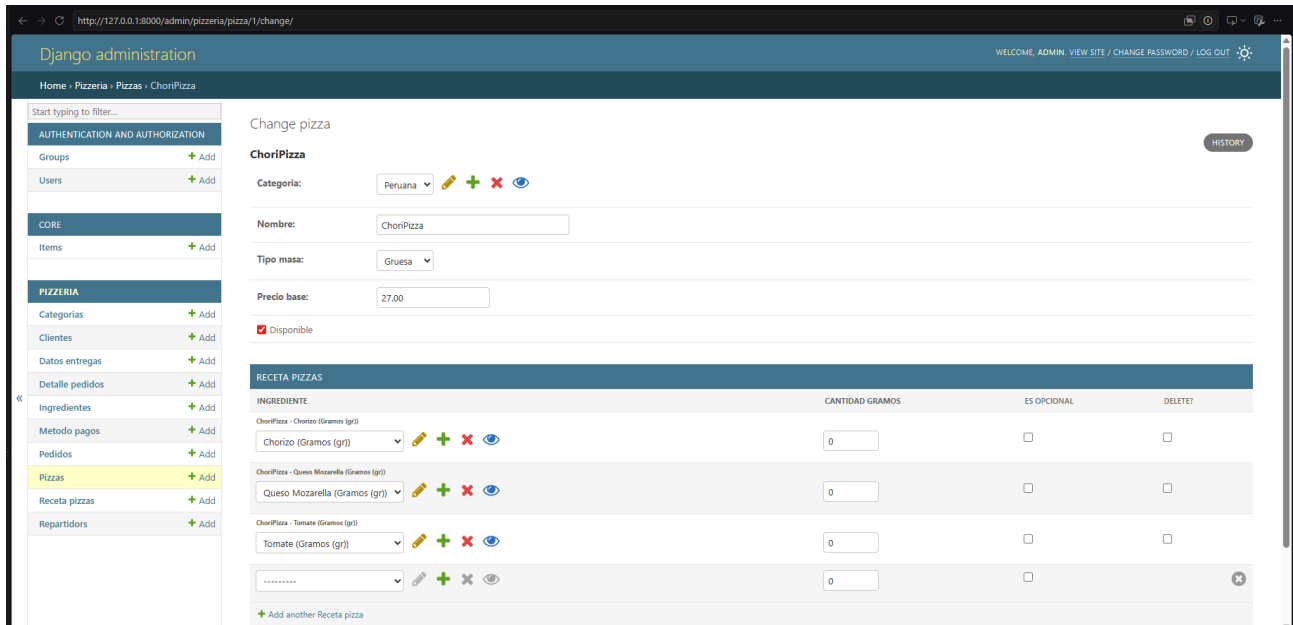
Implemente un StackedInline para la entidad de su relación OneToOneField, de modo que pueda verse y editarse dentro de la pantalla de la entidad principal.



Trata de la configuración de una clase basada en admin.StackedInline para la entidad con relación OneToOneField, permitiendo editar sus campos en un bloque apilado dentro del formulario del modelo principal.

## Ejercicio 7 — Exponer la relación N:M con TabularInline

Implemente un TabularInline para el modelo intermedio (through) de su relación ManyToManyField, mostrando sus atributos propios como columnas editables dentro de la pantalla de la entidad principal.

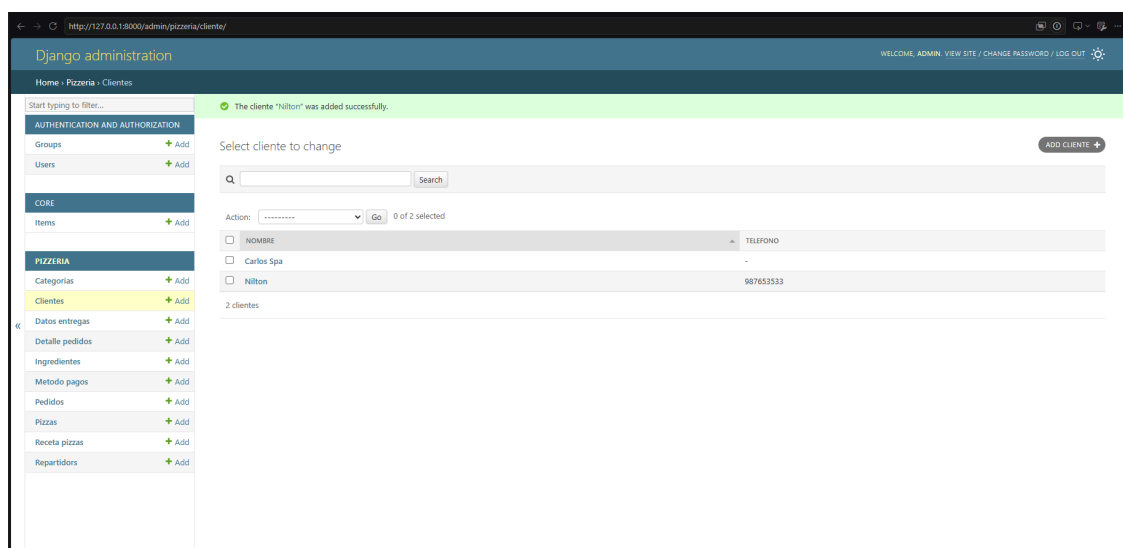


The screenshot shows the Django Admin interface for the 'Change pizza' form. The left sidebar contains a navigation menu with categories like AUTHENTICATION AND AUTHORIZATION, CORE, and PIZZERIA. The main content area is titled 'Change pizza' and shows the 'ChoriPizza' form. The form includes fields for 'Categoria' (Peruana), 'Nombre' (ChoriPizza), 'Tipo masa' (Gruesa), and 'Precio base' (27.00). Below these fields is a 'RECETA PIZZAS' section with a table of ingredients. The table has columns for 'INGREDIENTE', 'CANTIDAD GRAMOS', 'ES OPCIONAL', and 'DELETE?'. The ingredients listed are ChoriPizza - Chorizo (Gramos (gr)), ChoriPizza - Queso Mozzarella (Gramos (gr)), ChoriPizza - Tomate (Gramos (gr)), and an empty row for adding another ingredient. Each row has a quantity input field, a checkbox for 'ES OPCIONAL', and a delete button.

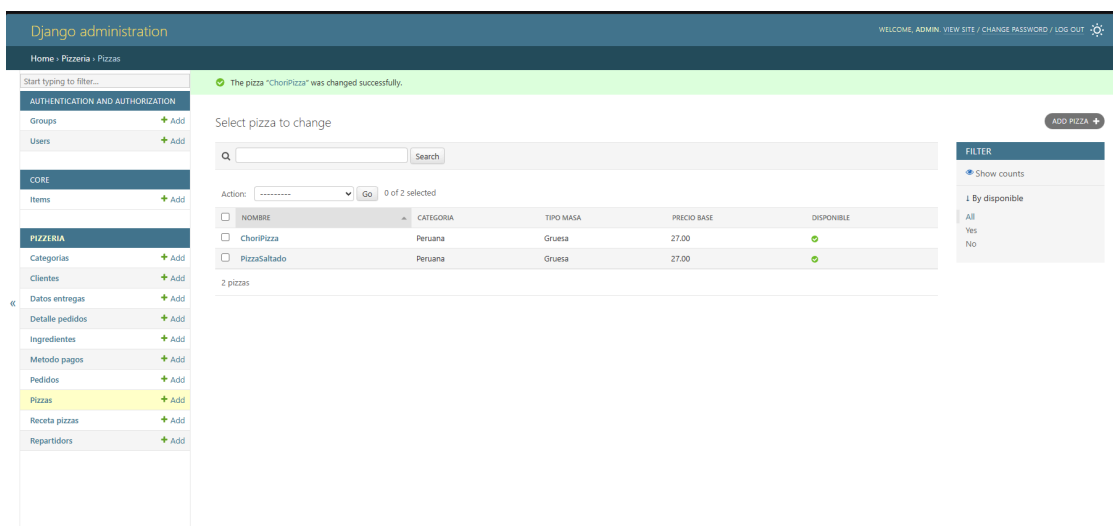
Consiste en la creación de una clase admin.TabularInline asociada al modelo intermedio de la relación ManyToManyField, lo que muestra una tabla editable con sus atributos propios (como cantidades) directamente en la pantalla de la entidad principal.

## Ejercicio 8 — Verificar el flujo completo en el Admin

Desde el panel del Admin, cree, edite y elimine al menos un registro de cada tipo de relación. Documente con capturas el recorrido completo y explique con sus palabras qué resuelve el Admin y qué seguiría necesitando una View y un Template propios para el usuario final.



The screenshot shows the Django Admin interface for the 'Clientes' list view. The left sidebar contains a navigation menu with categories like AUTHENTICATION AND AUTHORIZATION, CORE, and PIZZERIA. The main content area is titled 'Clientes' and shows a list of clients. A green message at the top indicates 'The cliente "Nilton" was added successfully.' Below the message is a 'Select cliente to change' section with a search bar and a list of clients. The clients listed are Carlos Spa and Nilton, with their phone numbers displayed. The bottom of the list shows '2 clientes'.



Django administration

Home - Pizzeria - Pizzas

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

CORE

Items + Add

PIZZERIA

Categorias + Add

Cientes + Add

Datos entregas + Add

Detalle pedidos + Add

Ingredientes + Add

Metodo pagos + Add

Pedidos + Add

Pizzas + Add

Receta pizzas + Add

Repartidores + Add

Select pizza to change

Q: [Search]

Action: [Go] 0 of 2 selected

| NOMBRE                                | CATEGORIA | TIPO MASA | PRECIO BASE | DISPONIBLE                           |
|---------------------------------------|-----------|-----------|-------------|--------------------------------------|
| <input type="checkbox"/> ChoriPizza   | Peruana   | Gruesa    | 27.00       | <span style="color: green;">●</span> |
| <input type="checkbox"/> PizzaSaltado | Peruana   | Gruesa    | 27.00       | <span style="color: green;">●</span> |

2 pizzas

ADD PIZZA +

FILTER

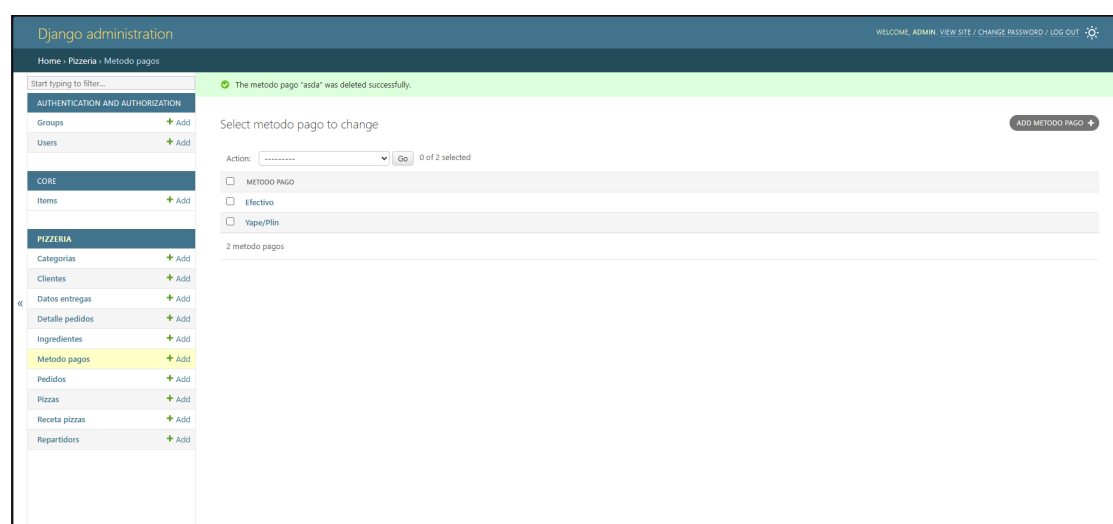
Show counts

By disponible

All

Yes

No



Django administration

Home - Pizzeria - Metodo pagos

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

CORE

Items + Add

PIZZERIA

Categorias + Add

Cientes + Add

Datos entregas + Add

Detalle pedidos + Add

Ingredientes + Add

Metodo pagos + Add

Pedidos + Add

Pizzas + Add

Receta pizzas + Add

Repartidores + Add

Select metodo pago to change

Q: [Search]

Action: [Go] 0 of 2 selected

| METODO PAGO                        |
|------------------------------------|
| <input type="checkbox"/> Efectivo  |
| <input type="checkbox"/> Yape/Plin |

2 metodo pagos

ADD METODO PAGO +

Abarca el recorrido operativo dentro del panel para crear, editar y eliminar registros en cada tipo de relación, analizando qué necesidades cubre el administrador nativo y justificando por qué aún se requiere una vista y plantilla propias para el usuario final.

## PARTE 2 — Administrar la investigación propia (ampliada en la Semana 4)

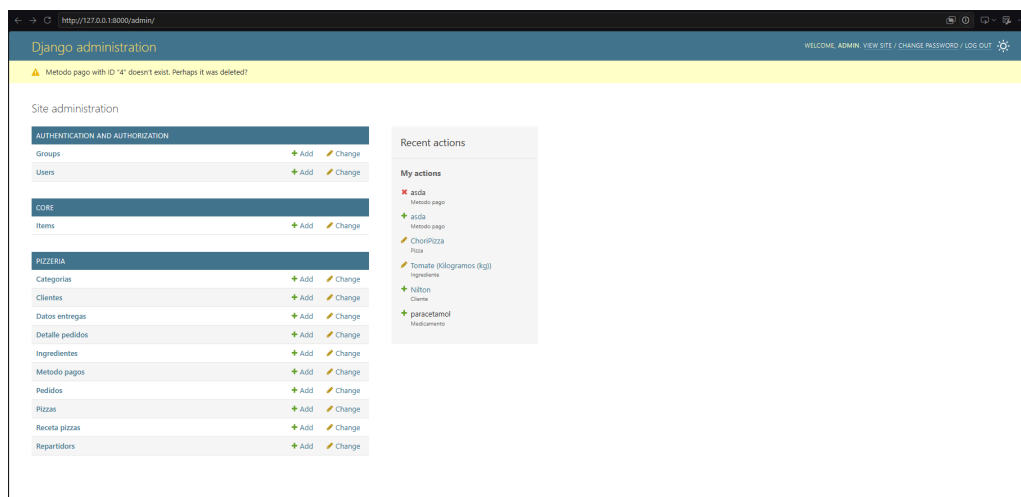
A partir de aquí trabajará sobre la base de datos ampliada de su investigación propia (la desarrollada en la Parte 2 del laboratorio de la Semana 4): las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es registrar y personalizar en el Django Admin ese mismo modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

## Ejercicio 9 — Registrar las siete entidades en el Admin

Registre en admin.py las siete entidades de su investigación propia (las cinco originales, la del 1:1 y el modelo intermedio del N:M) mediante `admin.site.register()`. Verifique que todas aparecen en el panel del Admin.

### Checklist antes de continuar — Parte 2

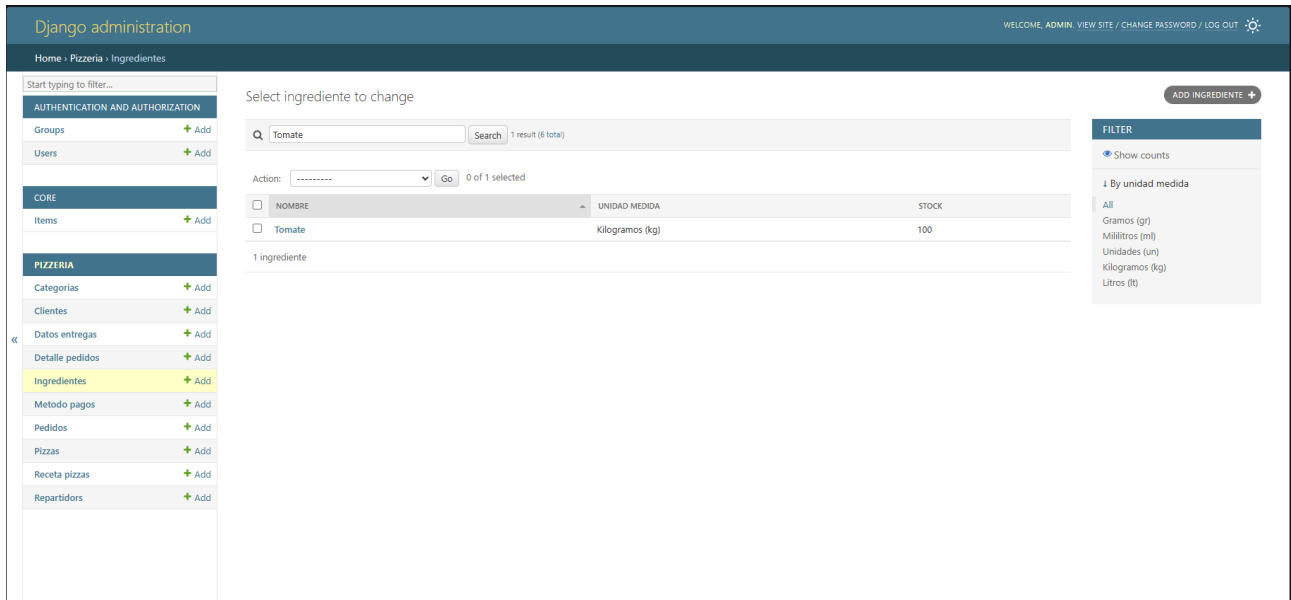
| Criterio                                                                                                                                           | Verificación                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| 1. Todas las entidades de tu investigación (las 7: 5 originales + entidad 1:1 + modelo intermedio N:M) están registradas en el Admin.              | Conteo directo en admin.py.           |
| 2. Al menos tres modelos usan una clase <code>ModelAdmin</code> (no solo <code>admin.site.register(Modelo)</code> sin personalizar).               | Código + captura del listado.         |
| 3. Al menos dos modelos tienen <code>list_display</code> configurado mostrando campos relevantes, no solo <code>__str__</code> .                   | Captura del listado en el Admin.      |
| 4. Al menos un modelo tiene <code>search_fields</code> configurado y funcionando.                                                                  | Captura de una búsqueda realizada.    |
| 5. Al menos un modelo tiene <code>list_filter</code> configurado y funcionando.                                                                    | Captura del panel de filtros.         |
| 6. La relación <code>OneToOneField</code> se expone con un <code>StackedInline</code> dentro de la entidad principal correspondiente.              | Captura del formulario con el Inline. |
| 7. El modelo intermedio de la relación <code>ManyToManyField</code> se expone con un <code>TabularInline</code> , mostrando sus atributos propios. | Captura del formulario con el Inline. |
| 8. Se realizó al menos una operación de creación, edición y eliminación completa desde el Admin (no solo lectura).                                 | Capturas del antes y el después.      |
| 9. Se explica por escrito qué resuelve el Admin para esta investigación y qué seguiría necesitando una View y un Template propios.                 | Párrafo de justificación.             |



Trata sobre el registro completo en admin.py de las siete entidades desarrolladas en la investigación propia (las 5 originales, la relación 1:1 y el modelo intermedio N:M), comprobando que todas estén visibles desde el panel.

## Ejercicio 10 — Personalizar ModelAdmin

Configure ModelAdmin en al menos tres de las siete entidades, con `list_display` mostrando los campos más relevantes de cada una. Agregue `search_fields` en al menos una y `list_filter` en al menos otra, eligiendo campos que tengan sentido para su problemática.

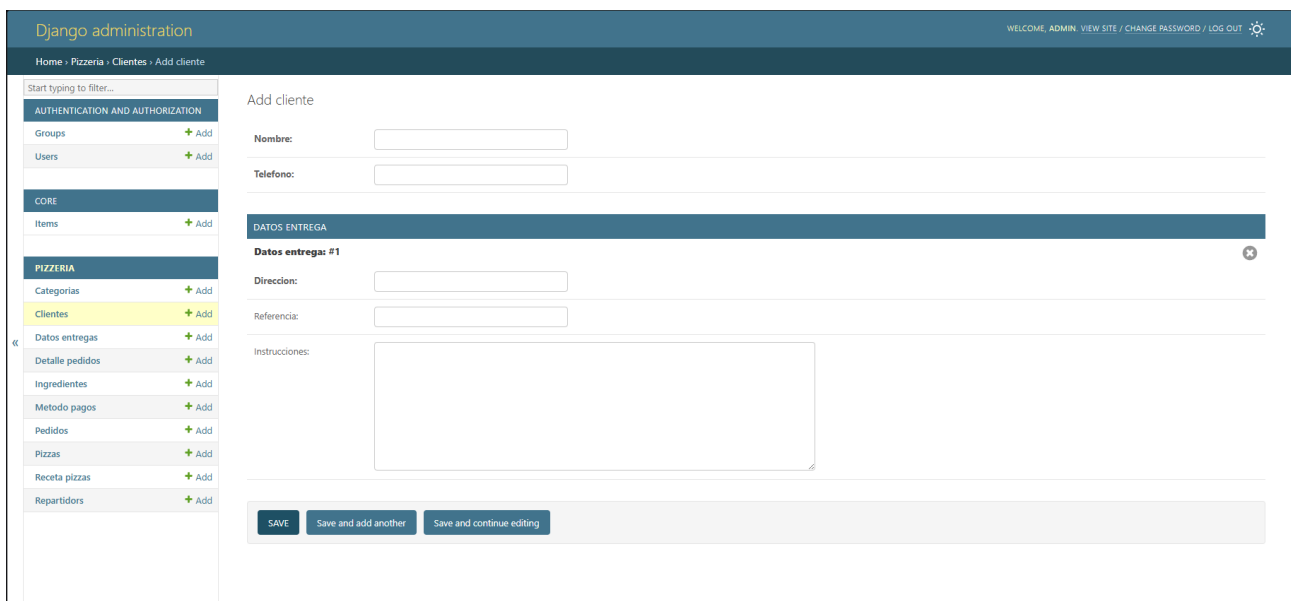


The screenshot shows the Django Admin interface for a Pizzeria application. The left sidebar lists the entities: AUTHENTICATION AND AUTHORIZATION (Groups, Users), CORE (Items), and PIZZERIA (Categorias, Clientes, Datos entregas, Detalle pedidos, Ingredientes, Metodo pagos, Pedidos, Pizzas, Receta pizzas, Repartidores). The 'Ingredientes' entity is selected and highlighted in yellow. The main content area shows the 'Select ingrediente to change' view. It includes a search bar with 'Tomate' entered, showing 1 result. Below the search bar is a table with columns: NOMBRE, UNIDAD MEDIDA, and STOCK. The table contains one row: 'Tomate' with 'Kilogramos (kg)' and '100'. To the right of the table is a 'FILTER' sidebar with a 'Show counts' option and a list of units: All, Gramos (gr), Mililitros (ml), Unidades (un), Kilogramos (kg), and Litros (lt).

Consiste en la personalización avanzada de al menos tres entidades del proyecto mediante ModelAdmin, aplicando `list_display` para detallar sus atributos clave, así como `search_fields` y `list_filter` según la lógica del negocio.

## Ejercicio 11 — StackedInline para la relación 1:1

Implemente un StackedInline para la entidad de su relación OneToOneField, de modo que se edite en la misma pantalla de la entidad principal a la que pertenece.

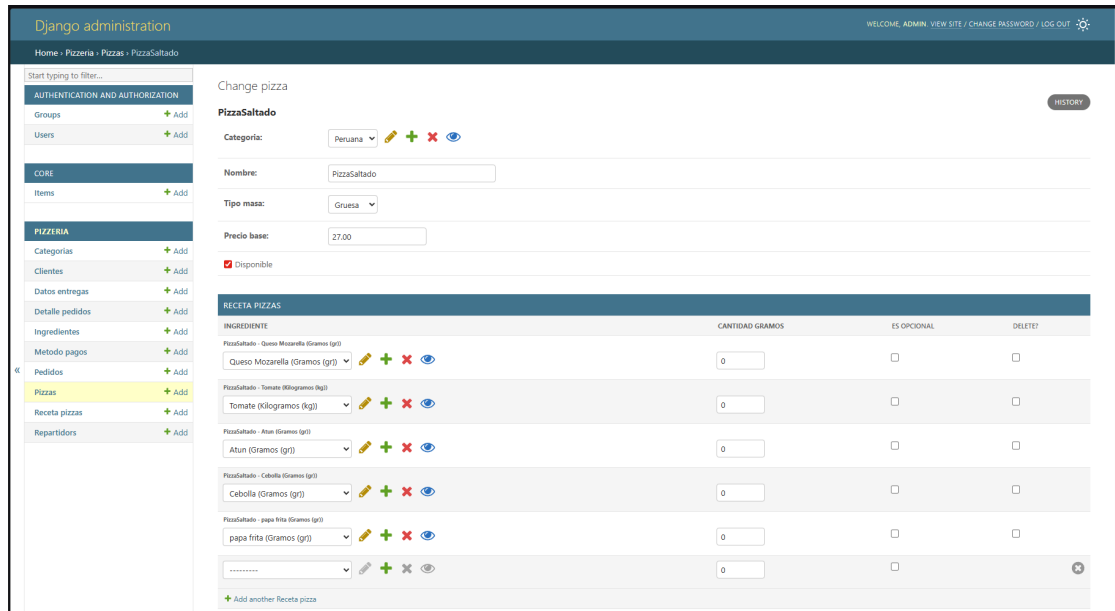


The screenshot shows the Django Admin interface for the 'Add cliente' form. The left sidebar is the same as in the previous screenshot. The main content area shows the 'Add cliente' form with fields for 'Nombre:' and 'Telefono:'. Below these fields is a 'DATOS ENTREGA' section, which is a StackedInline for the 'Datos entrega' entity. It contains a form with fields for 'Direccion:', 'Referencia:', and 'Instrucciones:'. At the bottom of the form are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

Aplica un StackedInline a la entidad 1:1 de la investigación propia para permitir la gestión de sus datos desde el formulario de creación o edición del modelo al que está vinculada.

## Ejercicio 12 — TabularInline para el modelo intermedio N:M

Implemente un TabularInline para el modelo intermedio de su relación ManyToManyField, mostrando como columnas editables los atributos propios definidos en la Semana 4 (por ejemplo cantidad, fecha o estado).



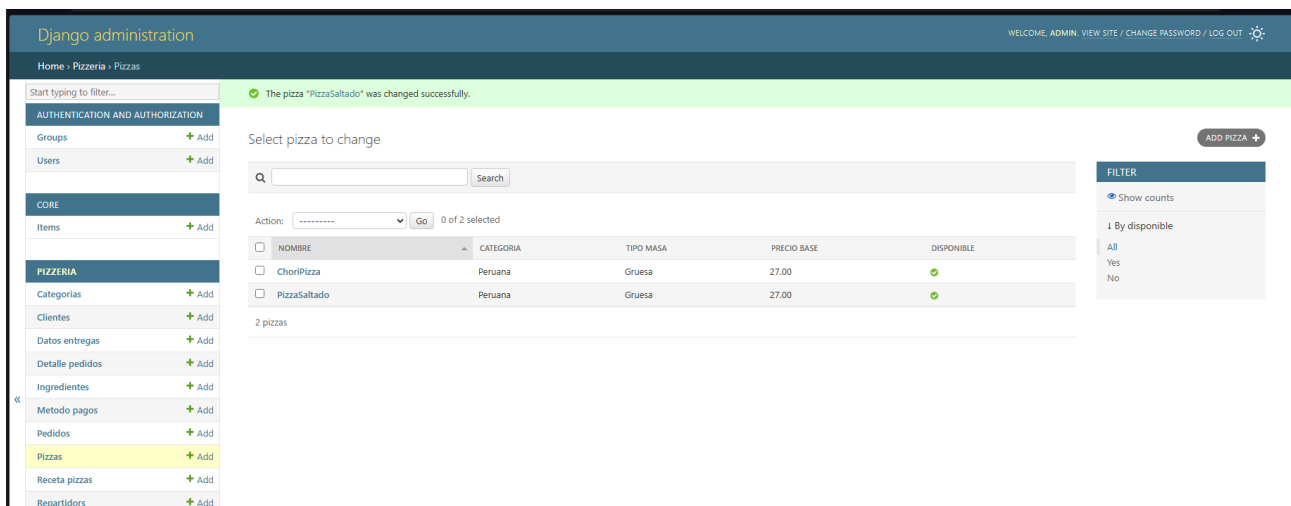
The screenshot shows the Django Admin interface for editing a 'PizzaSaltado' object. The left sidebar contains a menu with categories like AUTHENTICATION AND AUTHORIZATION, CORE, and PIZZERIA. The main form includes fields for 'Categoría', 'Nombre', 'Tipo masa', 'Precio base', and a 'Disponible' checkbox. Below these is a 'RECETA PIZZAS' section with a TabularInline table for ingredients.

| INGREDIENTE                                   | CANTIDAD GRAMOS | ES OPCIONAL              | DELETE?                  |
|-----------------------------------------------|-----------------|--------------------------|--------------------------|
| PizzaSaltado - Queso Mozzarella (Gramos (gr)) | 0               | <input type="checkbox"/> | <input type="checkbox"/> |
| PizzaSaltado - Tomate (Kilogramos (kg))       | 0               | <input type="checkbox"/> | <input type="checkbox"/> |
| PizzaSaltado - Atun (Gramos (gr))             | 0               | <input type="checkbox"/> | <input type="checkbox"/> |
| PizzaSaltado - Cebolla (Gramos (gr))          | 0               | <input type="checkbox"/> | <input type="checkbox"/> |
| PizzaSaltado - papa frita (Gramos (gr))       | 0               | <input type="checkbox"/> | <input type="checkbox"/> |
| .....                                         | 0               | <input type="checkbox"/> | <input type="checkbox"/> |

Implementa un TabularInline para la tabla intermedia de la investigación, permitiendo modificar columnas dinámicas (como cantidades, fechas o estados) directamente desde la pantalla de edición del elemento principal.

## Ejercicio 13 — Verificar el flujo completo

Desde el panel del Admin, cree, edite y elimine al menos un registro relacionado (usando los Inlines). Documente con capturas el antes y el después, y compruebe que los cambios persisten en SQLite.



The screenshot shows the Django Admin interface for selecting a pizza to edit. A green message at the top states 'The pizza "PizzaSaltado" was changed successfully.' Below is a search bar and a table of pizzas.

| INFORMACIÓN              | NOMBRE       | CATEGORIA | TIPO MASA | PRECIO BASE | DISPONIBLE                          |
|--------------------------|--------------|-----------|-----------|-------------|-------------------------------------|
| <input type="checkbox"/> | ChoriPizza   | Peruana   | Gruesa    | 27.00       | <input checked="" type="checkbox"/> |
| <input type="checkbox"/> | PizzaSaltado | Peruana   | Gruesa    | 27.00       | <input checked="" type="checkbox"/> |

2 pizzas

Consiste en realizar pruebas de integración en el panel creando, modificando y eliminando registros a través de los Inlines, verificando con capturas que los cambios se hayan guardado y persistan correctamente en la base de datos SQLite.

## Ejercicio 14 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo qué modelos quedaron registrados en el Admin y cómo se personalizó cada uno (ModelAdmin, list\_display, search\_fields, list\_filter, Inlines). Hagan commit y push al repositorio. Incluyan en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

<https://github.com/JoelQZ/DesarrolloAplicacionesEmpresariales/tree/main>

### Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de admin.py y el panel del Django Admin en el navegador).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

### Conclusiones:

- Eficiencia en la administración mediante Django Admin: La implementación del panel de administración nativo de Django permite gestionar de forma rápida, centralizada y segura las entidades del sistema, facilitando tareas de mantenimiento de datos para usuarios administradores sin necesidad de escribir SQL directamente.
- Optimización de la interfaz con ModelAdmin: La personalización a través de clases ModelAdmin mejora significativamente la usabilidad al incorporar atributos como list\_display, search\_fields y list\_filter, permitiendo búsquedas ágiles, filtrado de información por criterios relevantes y una visualización ordenada de las columnas clave de cada modelo.
- Mejora del flujo de trabajo con clases Inline: La integración de StackedInline (para relaciones 1:1) y TabularInline (para relaciones N:M con modelos intermedios) optimiza la experiencia de usuario, permitiendo crear, editar y eliminar registros relacionados en una sola pantalla sin necesidad de navegar por múltiples formularios.
- Persistencia y verificación de datos: La correcta ejecución y prueba del flujo completo (CRUD) desde el Admin confirma la integridad referencial y la persistencia automática de los datos en SQLite, validando que las restricciones y campos intermedios funcionen acorde a la lógica del negocio.
- Separación de roles en la arquitectura del sistema: Mientras el panel de Django Admin resuelve la administración técnica y el control interno de la base de datos, se concluye la necesidad de desarrollar vistas y plantillas personalizadas (Dashboard/POS) para la atención al cliente e interfaz operativa del usuario final.